ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 5

Կառուցվածքային տիպի զանգվածների օգտագործումը ֆունկցիաներում

N15. Շենքերի էներգաարդյունավետության գնահատում

Աշխատանքի նպատակը

- Սովորել օգտագործել կառուցվածքային տիպի զանգվածներ (`struct` զանգվածներ) C++ լեզվում և դրանց մշակման համար ստեղծել առանձին ֆունկցիաներ.
- Զարգացնել տվյալների հաշվարկման, որոնման, զտման և տեսակավորման գործնական հմտություններ .

Աշխատանքի պահանջը

- Ստեղծել `Building` կառուցվածք՝ հետևյալ դաշտերով՝ `address`, `area`, `monthlyEnergy`, `residents`, `year`
- Մուտքագրել 10 շենք.
- Ֆունկցիաների միջոցով իրականացնել .
    1. Հաշվել յուրաքանչյուր շենքի 1{ մ}^2-ի հաշվով էներգասպառումը
    2. Գտնել ամենացածր հարաբերական սպառում ունեցող շենքը ,
    3. Արտածել միջինից բարձր հարաբերական սպառում ունեցող շենքերը ,
    4. Հաշվել տրված տարուց հետո կառուցված շենքերի միջին սպառումը
    5. Դասավորել շենքերը ըստ 1{ մ}^2-ի էներգասպառման

Ընդհանուր պահանջներ

1. Յուրաքանչյուր խնդրում սահմանել առանձին `struct` տիպ և օգտագործել դաշտերի իմաստալից անգլերեն անուններ.
2. Մուտքագրումը, հաշվարկը, որոնումը կամ փոփոխումը կազմակերպել առնվազն երկու առանձին ֆունկցիայով .
3. Միայն ընթերցող ֆունկցիաներում կիրառել `const` հղում կամ `const` զանգված: Փոփոխող ֆունկցիաներում կիրառել հղում կամ ցուցիչ .
4. Ստուգել քանակների, գների, գնահատականների և այլ թվային տվյալների թույլատրելի սահմանները .
5. Դասակարգելիս փոխանակել ամբողջ կառուցվածքային տարրը .
6. Արտածումը ձևավորել ընթեռնելի վերնագրերով: Կոդը լրացնել կարճ մեկնաբանություններով

Տեսական մաս

Կառուցվածքային տիպերը (`struct`) թույլ են տալիս միավորել տարբեր տիպերի տվյալներ մեկ ընդհանուր օբյեկտի մեջ: Կառուցվածքների զանգվածների հետ

աշխատելիս յուրաքանչյուր շենքի տվյալները մշակվում են առանձին ֆունկցիաների միջոցով՝ ապահովելով կոդի մոդուլային և կառուցվածքային լինելը [cite: 25, 26].

```
#include <iostream>

#include <string>

#include <vector>

#include <iomanip>

using namespace std;

// Building կառուցվածք

struct Building {

    string address;       // Հասցե

    double area;          // Մակերես (մ²)

    double monthlyEnergy; // Ամսական էներգասպառում (կՎտ·ժ)

    int residents;        // Բնակիչների քանակ

    int year;             // Կառուցման տարեթիվ

};

// 1. Հաշվել յուրաքանչյուր շենքի 1 մ²-ի հաշվով էներգասպառումը

vector<double> calculateSpecificEnergy(const vector<Building>& buildings) {

    vector<double> specificEnergy(buildings.size());

    cout << "\n--- Յուրաքանչյուր շենքի 1 մ²-ի հաշվով էներգասպառումը ---\n";

    for (size_t i = 0; i < buildings.size(); ++i) {

        if (buildings[i].area > 0) {

            specificEnergy[i] = buildings[i].monthlyEnergy / buildings[i].area;

        } else {

            specificEnergy[i] = 0;

        }
```

```
        cout << "Շենք: " << buildings[i].address
             << " | Սպառում: " << fixed << setprecision(2) << specificEnergy[i] << " կՎտ·ժ/մ²\n";
    }
    return specificEnergy;
}
// 2. Գտնել ամենացածր հարաբերական սպառում ունեցող շենքը
void findMinSpecificEnergyBuilding(const vector<Building>& buildings) {
    if (buildings.empty()) return;
    size_t minIdx = 0;
    double minVal = buildings[0].monthlyEnergy / buildings[0].area;
    for (size_t i = 1; i < buildings.size(); ++i) {
        double val = buildings[i].monthlyEnergy / buildings[i].area;
        if (val < minVal) {
            minVal = val;
            minIdx = i;
        }
    }
    cout << "\n--- Ամենացածր հարաբերական սպառում ունեցող շենքը ---\n";
    cout << "Հասցե: " << buildings[minIdx].address
         << " | Սպառում: " << fixed << setprecision(2) << minVal << " կՎտ·ժ/մ²\n";
}
// 3. Արտածել միջինից բարձր հարաբերական սպառում ունեցող շենքերը
void printAboveAverageBuildings(const vector<Building>& buildings) {
    if (buildings.empty()) return;
```

```
    double totalSpecific = 0;

    vector<double> specific(buildings.size());

    for (size_t i = 0; i < buildings.size(); ++i) {

        specific[i] = buildings[i].monthlyEnergy / buildings[i].area;

        totalSpecific += specific[i];

    }

    double avgSpecific = totalSpecific / buildings.size();

    cout << "\n--- Միջինից բարձր հարաբերական սպառում ունեցող շենքերը (Միջին՝ " <<
fixed << setprecision(2) << avgSpecific << " կՎտ·ժ/մ²) ---\n";

    bool found = false;

    for (size_t i = 0; i < buildings.size(); ++i) {

        if (specific[i] > avgSpecific) {

            cout << "Հասցե: " << buildings[i].address

                << " | Սպառում: " << specific[i] << " կՎտ·ժ/մ²\n";

            found = true;

        }

    }

    if (!found) cout << "Այդպիսի շենքեր չկան:\n";

}

// 4. Հաշվել տրված տարուց հետո կառուցված շենքերի միջին սպառումը

void calculateAverageEnergyAfterYear(const vector<Building>& buildings, int targetYear) {

    double sum = 0;

    int count = 0;

    for (const auto& b : buildings) {

        if (b.year > targetYear) {
```

```
            sum += b.monthlyEnergy;
            count++;
        }
    }
    cout << "\n--- " << targetYear << " թվականից հետո կառուցված շենքերի միջին ամսական սպառումը ---\n";
    if (count > 0) {
        cout << "Միջին սպառում: " << fixed << setprecision(2) << (sum / count) << " կՎտ·ժ\n";
    } else {
        cout << "Այդպիսի շենքեր չեն հայտնաբերվել:\n";
    }
}
// 5. Դասավորել շենքերը ըստ 1 մ²-ի էներգասպառման (աճման կարգով)
void sortBuildingsBySpecificEnergy(vector<Building>& buildings) {
    for (size_t i = 0; i < buildings.size() - 1; ++i) {
        for (size_t j = 0; j < buildings.size() - i - 1; ++j) {
            double spec1 = buildings[j].monthlyEnergy / buildings[j].area;
            double spec2 = buildings[j + 1].monthlyEnergy / buildings[j + 1].area;
            if (spec1 > spec2) {
                swap(buildings[j], buildings[j + 1]);
            }
        }
    }
    cout << "\n--- Շենքերը դասավորված ըստ 1 մ²-ի էներգասպառման (աճման կարգով) ---\n";
```

```
    for (const auto& b : buildings) {
        double spec = b.monthlyEnergy / b.area;
        cout << "Հասցե: " << b.address
             << " | Մակերես: " << b.area << " մ²"
             << " | Սպառում: " << fixed << setprecision(2) << spec << " կՎտ·ժ/մ²\n";
    }
}
int main() {
    setlocale(LC_ALL, "Armenian");
    // Ստեղծում ենք 10 շենքից բաղկացած զանգված
    vector<Building> buildings = {
        {"Abovyan 10", 1200.5, 4500.0, 45, 2010},
        {"Mashtots 25", 850.0, 3200.0, 30, 2015},
        {"Komitas 42", 2000.0, 9500.0, 80, 2005},
        {"Baghramyan 5", 1500.0, 4800.0, 50, 2018},
        {"Northern Ave 1", 3000.0, 15000.0, 120, 2012},
        {"Tumanyan 14", 900.0, 3100.0, 35, 2019},
        {"Sayat-Nova 8", 1100.0, 4100.0, 40, 2008},
        {"Teryan 60", 1750.0, 7000.0, 65, 2016},
        {"Moskovyan 3", 950.0, 2900.0, 28, 2020},
        {"Arshakunyats 15", 2500.0, 11000.0, 100, 2014}
    };
    // Ֆունկցիաների կանչ
    calculateSpecificEnergy(buildings);
```

```
    findMinSpecificEnergyBuilding(buildings);

    printAboveAverageBuildings(buildings);

    calculateAverageEnergyAfterYear(buildings, 2015);

    sortBuildingsBySpecificEnergy(buildings);

    return 0;

}
```